Database Operations in SQL and MongoDB

1. Create a Database:

IN SQL:

- CREATE DATABASE mydb

IN MongoDB:

- use mydb

2. Create a Table (Collection in MongoDB):

IN SQL:

- CREATE TABLE users (
id INT PRIMARY KEY,
username VARCHAR(50),
age INT
)

IN MongoDB:

- db.createCollection("users");

3. INSERT DATA:

IN SQL:

- INSERT INTO users (id, username, age) values (1, 'David', 25);

IN MongoDB:

- db.users.insertOne({id:1, username: 'David', age:25});

4. FETCHING DATA:

IN SQL:

- SELECT * from users;

IN MongoDB:

- db.users.find();

IN SQL:

- SELECT * from users WHERE age > 21;

IN MongoDB:

- db.users.find({ age: {$gt : 21} });

5. UPDATING DATA:

IN SQL:

- UPDATE users SET age = 22 WHERE username = 'David';

IN MOngoDB:

- db.users.updateOne({username : 'David'}, { $set : {age : 22} });

6. DELETE DATA:

IN SQL:

- DELETE from users WHERE id = 1;

IN MongoDB:

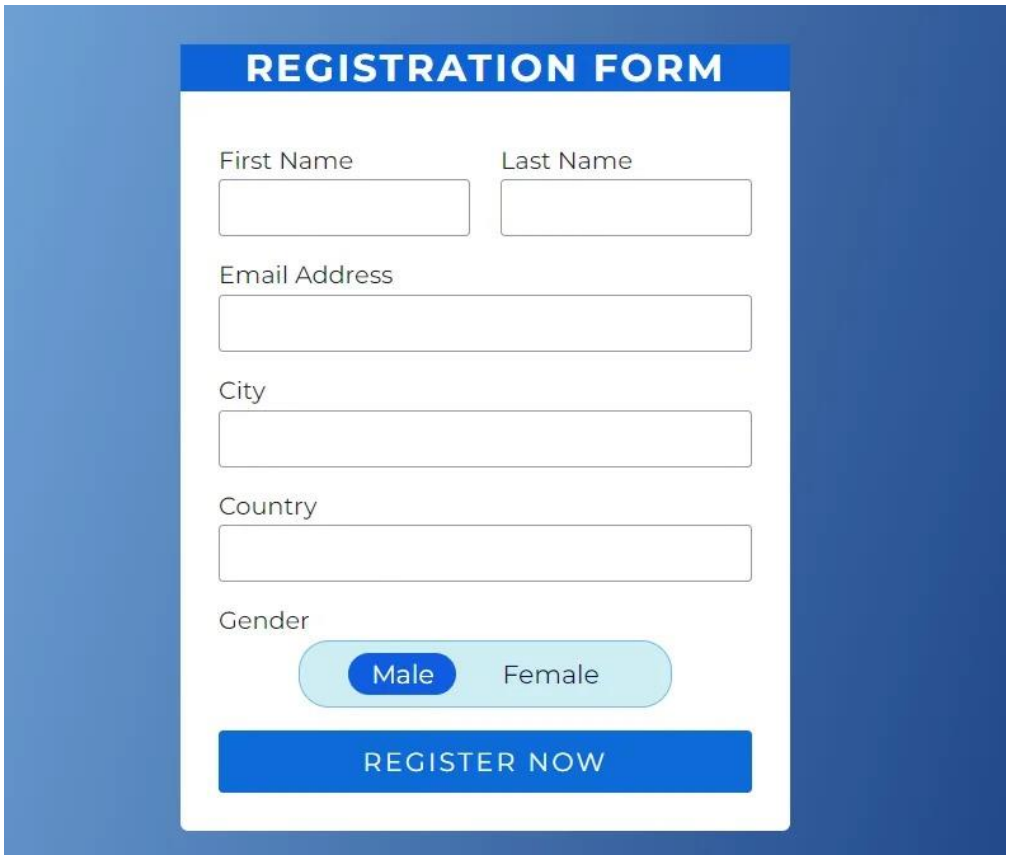
- db.users.deleteOne({ id : 1 });

MongoDB Compass:

- There are lots of the tools in the market that help to visualize data like MongoDB Compass, MongoDB Robo 3T
- mongodb://127.0.0.1:27017

DATA DESIGN and POSTMAN:

- Now in order to use Database we have to integrate MongoDB with our (NODE.JS/EXPRESS.JS) server.
- Now, there should be a FORM built on React JS or HTML or CSS to add Chef or Person details.
- Now currently we don't have a such frontend thing, so we are using POSTMAN for this.



- Every Frontend Application, collect Data and format it according to the Backend Requirements and then will send it to Backend APIs.

- Let's suppose creating Dummy Data for Person
- Each Person's details (like chef, owner, manager, waiter)

{

"name" : "David",

"age" : 28,

"work" : "chef",

"mobile" : "123-456-7890",

"email" : "david@gmail.com",

"address" : "123 Main St: City",

"salary" : 60000

}

- Each Menu's Details

{

"name" : "Mango Smoothie",

"price" : 12.9,

"taste" : "sweet",

"is_drink" : true,

"ingredients" : ["mango", "yogurt","honey"],

"address" : "123 Main St: City",

"num_sales" : 45

}

CONNECT MongoDB with (NODE.JS/EXPRESS.JS) Server

- Now to connect MongoDB with (NODE.JS/EXPRESS.JS) Server we need a **MongoDB Driver** (a set of programs)
- A MongoDB Driver is essential when connecting (NODE.JS/EXPRESS.JS) Server with MongoDB because it acts like a BRIDGE between (NODE.JS/EXPRESS.JS) Server and MongoDB Database
- MongoDB speaks its own language (protocol) to interact with the Database server
- (NODE.JS/EXPRESS.JS) communicates in JavaScript
- The Driver translates the JavaScript code from (NODE.JS/EXPRESS.JS) into a FORMAT that MongoDB can understand and vice versa.
- The Driver provides a set of Functions and Methods that make it easier to perform common Database Operations from your (NODE.JS/EXPRESS.JS) code.
- The Driver helps you handle errors that might occur during Database Interactions. It provides error codes, descriptions, and other details to help you troubleshoot issues.
- The most popular Driver is official MongoDB Node.js Driver, also known as the mongodb package.
    - **npm install mongodb**

MONGOOSE:

- Now but we are going to use Mongoose, rather than mongodb package.
- Mongoose is an **OBJECT DATA MODELING (ODM)** library for MongoDB and Node.js

- There are lots of reasons we prefer Mongoose rather than Native Official Driver (mongodb).
- Things are a lot easier here in Mongoose
- Mongoose is like a Translator between your NOde.js code and MongoDB. It makes working with the database smoother and easier.
- With Mongoose, you can define how your data should look, like making a Blueprint for your Document in Collection. It's like saying, "In our database, each person's information will have a name, age and email." This makes sure your data stays organized.
- Mongoose helps you make sure the data you put into the database is correct. It's like having someone check if you have written address correctly before sending the message.
- Very easy to query from the database.

→ **But if you are using mongodb Native Driver:**

- You need to write a lot of detailed instructions to make sure everything works correctly.
- Without Mongoose, your code might get messy and harder to understand.
- Since you need to handle many details yourself, it can take longer to finish your project.

Connect MongoDB Server with Express.js Server

- CREATE A FILE **db.js** IN YOUR ROOT FOLDER
- The **db.js** file that you have created is essentially responsible for establishing a connection between your Node.js Application and MongoDB database using the Mongoose library
- So let's import Mongoose Library

STEPS:

1) **Import Mongoose and Define the MongoDB URL:** In **db.js** file, you first import the Mongoose library and define the URL to your MongoDB database. This URL follows typically follows the format:

mongodb://<hostname>:<port>/<databaseName>

In your code, you have set the URL: **'mongodb://localhost:27017/myDatabase'**

where **myDatabase** is the name of your **MongoDB database**.

2) **Set Up the MongoDB Connection:** Next, you call **mongoose.connect()** to establish a connection to the MongoDB database using the URL. This step initializes the connection process but does not actually connect at this point.

3) **Access the Default Connection Object:** Mongoose maintains a **default connection object** representing the MongoDB connection. You retrieve this object using mongoose.connection, and you have stored it in the variable **'db'**. This object is what you will use to handle events and interact with the database.

4) **Define Event Listeners:** You define event listeners for the database connection using methods like **.on('connected', ...)**, **.on('error', ...)**, and **.on('disconnected', ...)**. These event listeners allow you to react to different states of the database connection.

5) **Start Listening for Events:** The code is set up to listen for events. When you call **mongoose.connect()**, Mongoose start the connection process.

- If the connection is successful, the 'connected' event is triggered, and you log a message indicating that "You are connected to MongoDB."
- If there's an error during connection process, the 'error' event is triggered, and you log an error message.
- Similarly, the 'disconnected' event can be useful for handling situations where the connection is lost.

6) **Export the Database Connection:** Finally, you **export the 'db' object**, which represents the MongoDB connection, so that you can import and use it in other parts of your Node.js application.

To sum it up, the db.js file acts as a central module that manages the connection to your MongoDB database using Mongoose. It sets up the connection, handles connection events, and Exports the connection object so that your Express.js Server (or other parts of your application) can use it to interact with the database. When your Server runs, it typically requires or imports this 'db.js' file to establish the database connection before handling HTTP requests.

db.js

db.js > ...

```
1
2   const mongoose = require('mongoose'); //import mongoose
3
4
5   //Define the Mongoose connection URL
6   const mongoURL = 'mongodb://localhost:27017/ucp';
7
8
9   //Set up mongoDB connection
10  mongoose.connect(mongoURL);
11
12
13
14  //Get the default connection
15  //Mongoose maintains a default connection object representing the mongoDB connection
16
17  const db = mongoose.connection;
18
19
20  //Define Event Listeners
21
22  db.on('connected', ()=> {console.log("Connected to MongoDB Server") });
23
24  db.on('error', (err)=> {console.log("MongoDB connection error: " + err) });
25
26  db.on('disconnected', ()=> {console.log("MongoDB Disconnected") });
27
28
29  //Export the database connection
30
31  module.exports = db;
32
```

What are Models or Schema?

- Models are like a Blueprint of our database
- It's a representation of a specific Collection in MongoDB. Like a person.
- Once you have defined a Model, you can **CREATE**, **READ**, **UPDATE** and **DELETE** Documents in the corresponding MongoDB **Collection**
- Mongoose allows you to define a Schema for your Documents within a Collection.
- Each Person's Detail like (chef, owner, manager, waiter)

{

"name" : "David",

"age" : 28,

"work" : "chef",

"mobile" : "123-456-7890",

"email" : "david@gmail.com",

"address" : "123 Main St: City",

"salary" : 60000

}

https://mongoosejs.com/docs/guide.html

- Parameters: Type, required, unique etc

Person Schema File: person.js

models > JS person.js > ...

```
const mongoose = require('mongoose');

// Define the Person Schema
const personSchema = new mongoose.Schema({
    name: {
        type: String,
        required: true
    },
    age: {
        type: Number
    },
    work: {
        type: String,
        enum: ['chef', 'owner', 'manager', 'waiter'],
        required: true
    },
    mobile: {
        type: String,
        required: true
    },
    email: {
        type: String,
        required: true,
        unique: true
    },
    address: {
        type: String
    },
    salary: {
        type: Number,  // Corrected "number" to "Number"
        required: true
    }
});

// Create a person model
const Person = mongoose.model('Person', personSchema, 'person');

// Export Person Model
module.exports = Person;

```

What is body-parser?

- **bodyParser** is a **middleware library** for Express.Js
- It is used to **Parse** and **Extract** the Body of incoming HTTP requests
- When a client (e.g., a Web Browser or a Mobile App) sends data to a server, it typically includes that data in the Body of an HTTP request.
- This data can be in various formats, such as **JSON**, **Form Data**, or **URL encoded data**. **bodyParser** helps parse and extract this data from the request so that you can work with it in your Express.js application
- **bodyParser** processes the request Body before it reaches your Route Handlers, making the parsed data available in the **req.body** for further processing.
- **bodyParser.json() automatically Parse the JSON data from the request Body and converts it into a JavaScript Object**. Which is then stored in the **req.body**
- Express.js uses lots of middleware and to use middleware we use the '**app.use()**'

→ **const bodyParser = require('body-parser');**

→ **app.use(bodyParser.json());**

SEND DATA FROM CLIENT TO SERVER AND SAVE IN DATABASE

- We need an Endpoint where the client sends data and data needs to be saved in the database
- We need a method called POST
- Now code the POST method to add the person
- If we send the random values as well Mongoose will not save random values other than predefined schema

```
newPerson.save((error, savedPerson)=>{
if(error) {
console.error('Error saving person: '+ error);
res.status(500).json({error:'Internal Server Errror'});
}
elase{
console.log('Data Saved');
res.status(201).json(savedPerson);
});
```

Async and Await

- Nowadays no one uses callback functions like we used in POST methods they look quite complex and do not give us code readability
- **What callback does:** callback is a function that is executed just after the execution of another main function, it means callback will wait untill its main function is not executed
- **Async and Await** are features in JavaScript that make it easier to work with asynchronous code, such as network requested, file system operations or database queries

Using 'try' and 'catch' block:

- The **try block** contains the code for creating a new Person document and saving it to the database using await newPerson.save().
- If an error occurs during any step, it is caught in the **catch block**, and an error response is sent with 500 Internal Server Error Status.

Index.js (Main Server File):

